

Laboratorium
Robotika dan Sistem Cerdas
Departemen Teknik Komputer
Institut Teknologi Sepuluh Nopember

Laporan Sementara
Praktikum Sistem Mikroprosesor dan Sistem Tertanam

PENGANTAR MIKROKONTROLER

Fauzian Ramadhan - 5024241080

2026

1 Pendahuluan

1.1 Latar Belakang

Perkembangan perangkat elektronik modern dan ekosistem *Internet of Things* (IoT) sangat bergantung pada sistem tertanam (*embedded system*) yang mengandalkan mikrokontroler sebagai pusat pemrosesan dan pengendalian perangkat keras. Dalam pengembangan sistem tertanam, praktisi dihadapkan pada keberagaman arsitektur perangkat keras—seperti mikrokontroler berbasis ARM Cortex-M4 pada *board* STM32 Blackpill (STM32F401CCU6) maupun arsitektur *dual-core* Xtensa LX6 dengan konektivitas nirkabel terintegrasi pada ESP32 DevKit. Selain keberagaman perangkat keras, terdapat pula perbedaan tingkat abstraksi perangkat lunak (*framework*), mulai dari *framework* Arduino yang berabstraksi tinggi hingga *framework native* seperti ESP-IDF yang dibangun langsung di atas sistem operasi waktu nyata (FreeRTOS).

Oleh karena itu, pelaksanaan Praktikum Modul 1: Pengantar Mikrokontroler ini memiliki urgensi yang sangat penting sebagai fondasi utama sebelum melangkah ke antarmuka sensor dan aktuator kompleks, komunikasi serial, hingga sistem kendali waktu nyata. Melalui praktikum ini, praktikan dibekali kemampuan mengonfigurasi ekosistem PlatformIO, melakukan *flashing* program pada STM32 Blackpill menggunakan ST-Link maupun pada ESP32 dengan ESP-IDF, mengimplementasikan konfigurasi resistor *pull-up* dan *pull-down* (eksternal maupun internal), menerapkan algoritma *debouncing* perangkat lunak, serta mengakses pin GPIO dalam ketiga modenya (*digital output*, *digital input*, dan *analog input*/ADC).

Permasalahan utama dalam perancangan perangkat keras dan sistem tertanam adalah bagaimana menghubungkan logika pemrograman perangkat lunak dengan respons fisik perangkat keras secara presisi, efisien, dan waktu nyata (real time). Tanpa pemahaman mendasar mengenai arsitektur mikrokontroler, konfigurasi register, manajemen pin General Purpose Input/Output (GPIO), serta pembacaan sinyal digital maupun analog, pengembang akan mengalami kesulitan dalam membangun sistem kendali yang andal dan bebas dari kegagalan fungsi (malfunction).

Oleh karena itu, praktikum Modul 1: Pengantar Mikrokontroler ini memiliki urgensi yang sangat tinggi sebagai fondasi awal sebelum mempelajari antarmuka sensor, aktuator motor, komunikasi serial, hingga sistem operasi waktu nyata (multitasking). Melalui praktikum ini, praktikan diharapkan mampu memahami arsitektur dasar mikrokontroler, menguasai lingkungan pengembangan terintegrasi (Integrated Development Environment/IDE), serta mengimplementasikan pengendalian dasar pin I/O sebagai langkah awal dalam merancang solusi teknologi sistem tertanam di dunia nyata.

1.2 Dasar Teori

1.2.1 PlatformIO dan VSCode sebagai Lingkungan Pengembangan

PlatformIO merupakan ekosistem pengembangan sistem tertanam (*embedded development ecosystem*) yang bersifat lintas platform (*cross-platform*) dan terintegrasi sebagai ekstensi di dalam Visual Studio Code (VSCode). Keunggulan utama PlatformIO adalah kemampuannya mendukung ratusan jenis

development board dan berbagai *framework* (seperti Arduino, STM32Cube/HAL, dan ESP-IDF) di dalam satu antarmuka yang seragam. Satu keping mikrokontroler yang sama (misalnya ESP32) dapat diprogram menggunakan *framework* Arduino yang sederhana maupun ESP-IDF yang bersifat *native* hanya dengan mengubah parameter konfigurasi proyek. Struktur direktori standar proyek PlatformIO meliputi:

- **platformio.ini**: Berkas konfigurasi utama yang mendefinisikan target *platform*, *board*, *framework*, serta protokol pengunggahan (**upload_protocol**) dan pelacakan kutu (**debug_tool**).
- **src**/: Direktori yang memuat berkas kode sumber utama program (seperti **main.cpp** atau **main.c**).
- **include**/: Direktori untuk menyimpan berkas kepala (*header files* **.h**).
- **lib**/: Direktori untuk menempatkan pustaka (*library*) tambahan yang bersifat lokal pada proyek.

1.2.2 STM32 Blackpill dan Framework Arduino (STM32duino)

STM32 Blackpill adalah papan pengembangan (*development board*) ringkas yang ditenagai oleh mikrokontroler 32-bit **STM32F401CCU6** berbasis prosesor **ARM Cortex-M4** dengan unit titik kambang (*Floating Point Unit*/FPU) dan frekuensi detak (*clock*) hingga 84 MHz. Mikrokontroler ini dilengkapi memori *Flash* sebesar 256 KB dan SRAM sebesar 64 KB. Pada papan Blackpill terdapat komponen bawaan berupa LED *onboard* yang terhubung pada pin **PC13** dengan logika **aktif-LOW** (menyala saat diberi logika *LOW*/0 V dan padam saat *HIGH*/3.3 V) serta tombol pengguna (*user button*) pada pin **PA0**. Karena Blackpill tidak memiliki sirkuit pemrogram (*programmer*) bawaan pada papan, proses pengunggahan program (*flashing*) dan *debugging* memerlukan perangkat eksternal **ST-Link V2** melalui protokol **SWD (Serial Wire Debug)** yang menggunakan empat jalur koneksi: **SWDIO** (jalur data), **SWCLK** (jalur *clock*), **GND** (referensi *ground*), dan **3.3V** (catu daya target).

Melalui dukungan inti **STM32duino**, papan STM32 dapat diprogram menggunakan *framework* Arduino yang menyediakan fungsi standar berabstraksi tinggi seperti **pinMode()**, **digitalWrite()**, dan **delay()**. Di balik layar, STM32duino dibangun di atas pustaka resmi **HAL (Hardware Abstraction Layer)** milik STMicroelectronics yang secara otomatis menangani pengaktifan *clock peripheral* GPIO dan konfigurasi struktur register. Ciri khas STM32duino adalah dukungannya terhadap penamaan makro pin yang langsung merepresentasikan label port fisik pada papan, seperti **PC13** atau **PA0**.

1.2.3 ESP32 dan Framework ESP-IDF

ESP32 (pada papan ESP32 DevKit v1) adalah mikrokontroler 32-bit *dual-core* berbasis arsitektur **Xtensa LX6** yang mengintegrasikan modul komunikasi nirkabel WiFi dan Bluetooth secara langsung di dalam satu keping (*System on Chip*). Berbeda dengan Blackpill, papan ESP32 DevKit telah dilengkapi jembatan konverter *USB-to-Serial* bawaan (seperti CP2102 atau CH340) sehingga dapat diprogram langsung melalui kabel USB tanpa memerlukan *programmer* eksternal.

Selain dapat diprogram menggunakan *framework* Arduino, ESP32 memiliki kerangka kerja resmi dan *native* dari Espressif Systems yaitu **ESP-IDF (Espressif IoT Development Framework)** yang dibangun langsung di atas sistem operasi waktu nyata **FreeRTOS**. Perbedaan mendasar antara kedua *framework* tersebut dirangkum sebagai berikut:

- **Titik Masuk Program (Entry Point):** *Framework* Arduino menggunakan pasangan fungsi **setup()** dan **loop()**, sedangkan ESP-IDF menggunakan fungsi **app_main(void)** yang dieksekusi langsung sebagai sebuah *task* FreeRTOS.
- **Manajemen Pewaktuan (Timing/Delay):** *Framework* Arduino menggunakan **delay(ms)**, sedangkan ESP-IDF menggunakan fungsi FreeRTOS **vTaskDelay(pdMS_TO_TICKS(ms))** yang bekerja berbasis *tick* penjadwalan sehingga tidak memblokir eksekusi *task* lain pada sistem.
- **Pengendalian GPIO dan Abstraksi:** Pada ESP-IDF, konfigurasi pin dilakukan melalui pustaka **driver/gpio.h** menggunakan fungsi seperti **gpio_reset_pin()**, **gpio_set_direction()**, dan **gpio_set_level()**. Meskipun memiliki tingkat abstraksi rendah–menengah yang lebih kompleks dibanding Arduino, ESP-IDF memberikan kendali penuh terhadap fitur tingkat rendah perangkat keras serta manajemen objek FreeRTOS (*task*, *semaphore*, dan *queue*).

2 Tugas Pendahuluan

Bagian ini berisi jawaban dari tugas pendahuluan yang telah anda kerjakan, beserta penjelasan dari jawaban tersebut

1. **Mengapa board STM32 Blackpill memerlukan programmer eksternal (ST-Link) untuk di-flash, sedangkan ESP32 dapat diprogram langsung melalui kabel USB? Kaitkan jawaban Anda dengan keberadaan chip USB-to-Serial.**
   Jawaban dan Penjelasan:

   - **STM32 Blackpill:** Papan pengembangan ini tidak dilengkapi dengan sirkuit pemrogram (*programmer*) atau jembatan komunikasi USB-to-Serial secara fisik di atas papannya. Untuk mentransfer kode mesin (*binary/hex*) dari PC ke memori *Flash* mikrokontroler STM32F401CCU6, diperlukan perangkat keras pemrogram eksternal **ST-Link V2** yang terhubung melalui antarmuka protokol **SWD (Serial Wire Debug)** menggunakan empat jalur pin utama (`SWDIO`, `SWCLK`, `GND`, dan `3.3V`).
   - **ESP32 DevKit:** Papan ESP32 sudah mengintegrasikan *chip* jembatan konverter **USB-to-Serial** bawaan (seperti CP2102 atau CH340) di atas papannya. Ketika kabel USB dihubungkan ke PC, *chip* tersebut menerjemahkan sinyal komunikasi serial USB dari komputer menjadi sinyal UART mikrokontroler ESP32, serta dilengkapi rangkaian otomatis untuk mengontrol pin *boot/EN* sehingga mikrokontroler siap menerima program baru tanpa perangkat keras tambahan.

2. **Jelaskan perbedaan kondisi logika (HIGH/LOW) pada konfigurasi pull-up dan pull-down resistor, baik saat tombol tidak ditekan maupun saat ditekan.**
   Jawaban dan Penjelasan:

   - **Konfigurasi Pull-up:** Pin GPIO dihubungkan ke sumber tegangan V_{CC} (3.3 V) melalui resistor, dan tombol menghubungkan pin ke `GND` saat ditekan.

  - *Saat tidak ditekan:* Pin terbaca stabil pada logika **HIGH** (karena ditarik ke V_{CC}).
  - *Saat ditekan:* Pin terhubung langsung ke `GND` sehingga terbaca **LOW** (logika *active-low*).
- **Konfigurasi Pull-down:** Pin GPIO dihubungkan ke `GND` melalui resistor, dan tombol menghubungkan pin ke V_{CC} (3.3 V) saat ditekan.
  - *Saat tidak ditekan:* Pin terbaca stabil pada logika **LOW** (karena ditarik ke `GND`).
  - *Saat ditekan:* Pin terhubung ke V_{CC} sehingga terbaca **HIGH** (logika *active-high*).

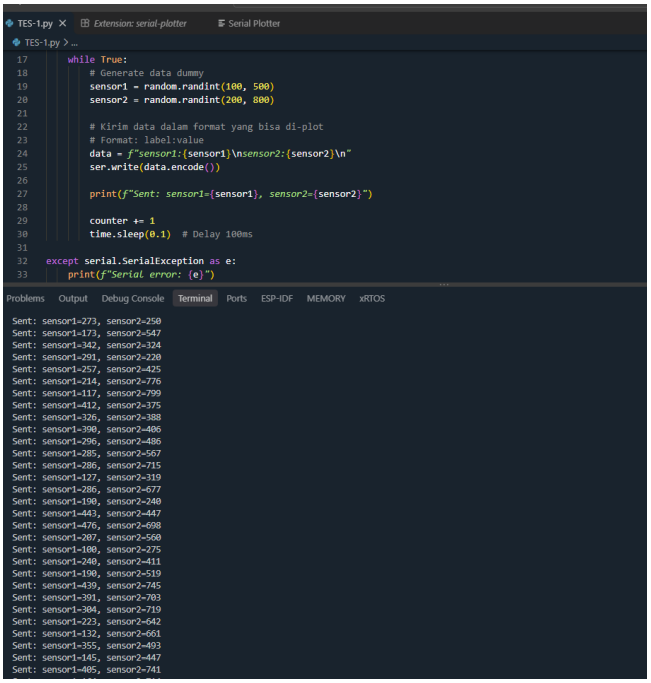
3. **Jelaskan fenomena bouncing pada tombol mekanik — mengapa hal ini bisa terjadi — dan sebutkan minimal dua pendekatan untuk mengatasinya.**
   Jawaban dan Penjelasan:

   - **Penyebab Bouncing:** Tombol mekanik menggunakan kontak lempeng logam pegas. Ketika tombol ditekan atau dilepaskan, kontak logam tersebut mengalami tumbukan elastis mikroskopis dan getaran fisik sesaat sebelum benar-benar mantap bersentuhan. Hal ini memicu rentetan transisi sinyal elektrik *HIGH–LOW* berfrekuensi tinggi dalam waktu beberapa milidetik.
   - **Pendekatan Mengatasinya:**
     (a) **Pendekatan Perangkat Lunak (Software Debouncing):** Menggunakan algoritma penundaan waktu berbasis fungsi waktu internal seperti `millis()` (mengecek apakah status sinyal bertahan stabil melewati batas waktu tunda tertentu, misal `DEBOUNCE_DELAY` 20–50 ms).
     (b) **Pendekatan Perangkat Keras (Hardware Debouncing):** Menambahkan komponen kapasitor secara paralel pada rangkaian tombol untuk meredam lonjakan tegangan transien, atau menggunakan rangkaian gerbang logika *Schmitt trigger* / filter RC.

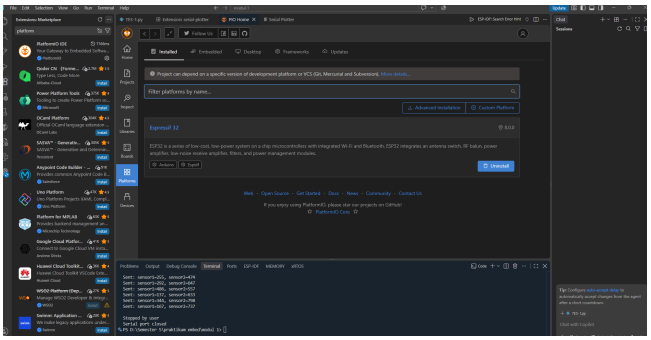
4. **Bagaimana ADC (Analog-to-Digital Converter) mengubah sinyal analog menjadi nilai digital? Jelaskan pula pengaruh resolusi bit ADC terhadap presisi hasil pembacaan.**
   Jawaban dan Penjelasan:

   - **Prinsip Kerja ADC:** Periferal ADC melakukan proses pencuplikan (*sampling*), penahanan nilai tegangan (*sample and hold*), kuantisasi, dan pengkodean biner. Tegangan analog kontinu dicuplik dan dibandingkan terhadap tegangan referensi (V_{ref}), lalu dipetakan ke dalam bilangan bulat diskrit.
   - **Pengaruh Resolusi Bit (n-bit):** Resolusi bit menentukan jumlah tingkatan diskrit (2^n) yang tersedia untuk membagi rentang tegangan referensi. Pada ESP32 dengan ADC beresolusi **12-bit**, rentang tegangan dibagi menjadi $2^{12} = 4096$ level (0 hingga 4095). Semakin tinggi resolusi bit ADC (misalnya 12-bit dibanding 10-bit), ukuran langkah kuantisasi (*step size*) menjadi semakin kecil sehingga presisi dan kehalusan hasil pembacaan tegangan analog semakin meningkat.

5. **Lakukan set-up untuk tools: vscode, ekstensi platform-io (dengan framework esp-idf dan stm32), dan ekstensi serial-plotter pada laptop kalian, berikan screenshoot bahwa laptop kalian sudah ter-set-up dengan baik.**



Gambar 1: Tes Serial Plotter



Gambar 2: set up platform IO